

Transverse-Field Ising Model Simulation: Demonstrating better performance with Q-CTRL's Qiskit Functions

Usage estimate: 2 minutes on a Heron r2 processor. (NOTE: This is an estimate only. Your runtime may vary.)

Background

The Transverse-Field Ising Model (TFIM) is important for studying quantum magnetism and phase transitions. It describes a set of spins arranged on a lattice, where each spin interacts with its neighbors while also being influenced by an external magnetic field that drives quantum fluctuations.

A common approach to simulate this model is to use Trotter decomposition to approximate the time evolution operator, constructing circuits that alternate between single-qubit rotations and entangling two-qubit interactions. However, this simulation on real hardware is challenging due to noise and decoherence, leading to deviations from the true dynamics. To overcome this, we use Q-CTRL's Fire Opal error suppression and performance management tools, offered as a Qiskit Function (see the [Fire Opal documentation](#)). Fire Opal automatically optimizes circuit execution by applying dynamical decoupling, advanced layout, routing, and other error suppression techniques, all aimed at reducing noise. With these improvements, the hardware results align more closely with noiseless simulations, and thus we can study TFIM magnetization dynamics with higher fidelity.

In this tutorial we will:

- Build the TFIM Hamiltonian on a graph of connected spin triangles
- Simulate time evolution with Trotterized circuits at different depths
- Compute and visualize single-qubit magnetizations $\langle Z_i \rangle$ over time
- Compare baseline simulations with results from hardware runs using Q-CTRL's Fire Opal performance management

Overview

The Transverse-field Ising Model (TFIM) is a quantum spin model that captures essential features of quantum phase transitions. The Hamiltonian is defined as:

$$H = -J \sum_i Z_i Z_{i+1} - h \sum_i X_i$$

where Z_i and X_i are Pauli operators acting on qubit i , J is the coupling strength between neighboring spins, and h is the strength of the transverse magnetic field. The first term represents classical ferromagnetic interactions, while the second introduces quantum fluctuations through the transverse field. To simulate TFIM dynamics, you use a Trotter decomposition of the unitary evolution operator e^{-iHt} , implemented through layers of RX and RZZ gates based on a custom graph of connected spin triangles. The simulation explores how magnetization $\langle Z \rangle$ evolves with increasing Trotter steps.

The performance of the proposed TFIM implementation is assessed by comparing noiseless simulations with noisy backends. Fire Opal's enhanced execution and error suppression features are used to mitigate the effect of noise in real hardware, yielding more reliable estimates of spin observables like $\langle Z_i \rangle$ and correlators $\langle Z_i Z_j \rangle$.

Requirements

Before starting this tutorial, be sure you have the following installed:

- Qiskit SDK v1.4 or later, with visualization support (`pip install 'qiskit[visualization]'`)
- Qiskit Runtime v0.40 or later (`pip install qiskit-ibm-runtime`)
- Qiskit Functions Catalog v0.9.0 (`pip install qiskit-ibm-catalog`)
- Fire Opal SDK v9.0.2 or later (`pip install fire-opal`)
- Q-CTRL Visualizer v8.0.2 or later (`pip install qctrl-visualizer`)

Setup

```

1  from qiskit.transpiler.preset_passmanagers import
2  from qiskit import QuantumCircuit
3  from qiskit_ibm_catalog import QiskitFunctionsCatalog
4  from qiskit_ibm_runtime import QiskitRuntimeService
5  from qiskit_ibm_runtime import SamplerV2 as Sampler
6  from qiskit.quantum_info import SparsePauliOp
7  from qiskit_aer import AerSimulator
8
9  import numpy as np
10 import networkx as nx
11 import matplotlib.pyplot as plt
12 import qctrlvisualizer as qv

```

```

1  # Set credentials.
2  token = "YOUR_IBM_CLOUD_API_KEY"
3  instance = "YOUR_IBM_CRN"
4
5  catalog = QiskitFunctionsCatalog(
6      channel="ibm_quantum_platform", instance=instance
7  )
8
9  # Access Function
10 perf_mgmt = catalog.load("q-ctrl/performance-managemei

```

Step 1: Map classical inputs to a quantum problem

Generate TFIM graph

We begin by defining the lattice of spins and the couplings between them. In this tutorial, the lattice is constructed from connected triangles arranged in a linear chain. Each triangle consists of three nodes connected in a closed loop, and the chain is formed by linking one node of each triangle to the previous triangle.

The helper function `connected_triangles_adj_matrix` builds the adjacency matrix for this structure. For a chain of n triangles, the resulting graph contains $2n + 1$ nodes.

```

1  def connected_triangles_adj_matrix(n):
2      """
3      Generate the adjacency matrix for 'n' connected triangles.
4      """
5      num_nodes = 2 * n + 1
6      adj_matrix = np.zeros((num_nodes, num_nodes), dtype=int)
7
8      for i in range(n):
9          a, b, c = i * 2, i * 2 + 1, i * 2 + 2  # Nodes in triangle
10
11         # Connect the three nodes in a triangle
12         adj_matrix[a, b] = adj_matrix[b, a] = 1
13         adj_matrix[b, c] = adj_matrix[c, b] = 1
14         adj_matrix[a, c] = adj_matrix[c, a] = 1
15
16         # If not the first triangle, connect to the previous one
17         if i > 0:
18             adj_matrix[a, a - 1] = adj_matrix[a - 1, a] = 1
19
20     return adj_matrix

```

To visualize the lattice we just defined, we can plot the chain of connected triangles and label each node. The function below builds the graph for a chosen number of triangles and displays it.

```

1  def plot_triangle_chain(n, side=1.0):
2      """
3      Plot a horizontal chain of n equilateral triangles.
4      Baseline: even nodes (0,2,4,...,2n) on y=0
5      Apexes: odd nodes (1,3,5,...,2n-1) above the midpoint
6      """
7      # Build graph
8      A = connected_triangles_adj_matrix(n)
9      G = nx.from_numpy_array(A)
10
11     h = np.sqrt(3) / 2 * side
12     pos = {}
13
14     # Place baseline nodes
15     for k in range(n + 1):
16         pos[2 * k] = (k * side, 0.0)
17
18     # Place apex nodes
19     for k in range(n):
20         x_left = pos[2 * k][0]
21         x_right = pos[2 * k + 2][0]
22         pos[2 * k + 1] = ((x_left + x_right) / 2, h)
23
24     # Draw
25     fig, ax = plt.subplots(figsize=(1.5 * n, 2.5))
26     nx.draw(
27         G,
28         pos,
29         ax=ax,
30         with_labels=True,
31         font_size=10,
32         font_color="white",
33         node_size=600,
34         node_color=qv.QCTRL_STYLE_COLORS[0],
35         edge_color="black",
36         width=2,
37     )
38     ax.set_aspect("equal")
39     ax.margins(0.2)
40     plt.show()
41
42     return G, pos

```

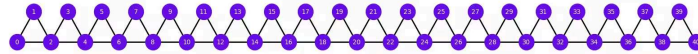
For this tutorial we will use a chain of 20 triangles.

```

1  n_triangles = 20
2  n_qubits = 2 * n_triangles + 1
3  plot_triangle_chain(n_triangles, side=1.0)
4  plt.show()

```

Output:



Coloring graph edges

To implement the spin–spin coupling, it is useful to group edges that do not overlap. This allows us to apply two-qubit gates in parallel. We can do this with a simple edge-coloring procedure [1], which assigns a color to each edge so that edges meeting at the same node are placed in different groups.

```

1  def edge_coloring(graph):
2      """
3      Takes a NetworkX graph and returns a list of lists
4      the edges assigned the same color.
5      """
6      line_graph = nx.line_graph(graph)
7      edge_colors = nx.coloring.greedy_color(line_graph)
8
9      color_groups = {}
10     for edge, color in edge_colors.items():
11         if color not in color_groups:
12             color_groups[color] = []
13             color_groups[color].append(edge)
14
15     return list(color_groups.values())

```

Step 2: Optimize problem for quantum hardware execution

Generate Trotterized circuits on spin graphs

To simulate the dynamics of the TFIM, we construct circuits that approximate the time evolution operator.

$$U(t) = e^{-iHt}, \quad \text{where} \quad H = -J \sum_{\langle i,j \rangle} Z_i Z_j - h \sum_i X_i.$$

We use a second-order Trotter decomposition:

$$e^{-iH\Delta t} \approx e^{-iH_X\Delta t/2} e^{-iH_Z\Delta t} e^{-iH_X\Delta t/2},$$

where $H_X = -h \sum_i X_i$ and $H_Z = -J \sum_{\langle i,j \rangle} Z_i Z_j$.

- The H_X term is implemented with layers of `RX` rotations.
- The H_Z term is implemented with layers of `RZZ` gates along the edges of the interaction graph.

The angles of these gates are determined by the transverse field h , the coupling constant J , and the time step Δt . By stacking multiple Trotter steps, we generate circuits of increasing depth that approximate the system's dynamics. The functions `generate_tfim_circ_custom_graph` and `trotter_circuits` construct a Trotterized quantum circuit from an arbitrary spin interaction graph.

```

1  def generate_tfim_circ_custom_graph(
2      steps, h, J, dt, psi0, graph: nx.graph.Graph, meas
3  ):
4      """
5      Generate a second order trotter of the form  $e^{(a+b)}$ 
6       $e^{\{-i H t\}}$  where the Hamiltonian  $H = -J \sum_i Z_i$ 
7
8      steps: Number of trotter steps
9      theta_x: Angle for layer of X rotations
10     theta_zz: Angle for layer of ZZ rotations
11     theta_x: Angle for second layer of X rotations
12     J: Coupling between nearest neighbor spins
13     h: The transverse magnetic field strength
14     dt: t/total_steps
15     psi0: initial state (assumed to be prepared in the
16     meas_basis: basis to measure all correlators in
17
18     This is a second order trotter of the form  $e^{(a+b)}$ 
19     """
20     theta_x = h * dt
21     theta_zz = -2 * J * dt
22     nq = graph.number_of_nodes()
23     color_edges = edge_coloring(graph)
24     circ = QuantumCircuit(nq, nq)
25     # Initial state, for typical cases in the computat
26     for i, b in enumerate(psi0):
27         if b == "1":
28             circ.x(i)
29     # Trotter steps
30     for step in range(steps):
31         for i in range(nq):
32             circ.rx(theta_x, i)
33         if mirror:
34             color_edges = [sublist[::-1] for sublist :
35         for edge_list in color_edges:
36             for edge in edge_list:
37                 circ.rzz(theta_zz, edge[0], edge[1])
38         for i in range(nq):
39             circ.rx(theta_x, i)
40
41     # some typically used basis rotations
42     if meas_basis == "X":
43         for b in range(nq):
44             circ.h(b)
45     elif meas_basis == "Y":
46         for b in range(nq):
47             circ.sdg(b)
48             circ.h(b)
49
50     for i in range(nq):
51         circ.measure(i, i)
52

```

```

53     return circ
54
55
56 def trotter_circuits(G, d_ind_tot, J, h, dt, meas_bas:
57     """
58     Generates a sequence of Trotterized circuits, each
59     Given a spin interaction graph and Hamiltonian pa
60     a list of circuits with 1 to d_ind_tot Trotter ste
61
62     G: Graph defining spin interactions (edges = ZZ co
63     d_ind_tot: Number of Trotter steps (maximum depth)
64     J: Coupling between nearest neighboring spins
65     h: Transverse magnetic field strength
66     dt: (t / total_steps
67     meas_basis: Basis to measure all correlators in
68     mirror: If True, mirror the Trotter layers
69     """
70     qubit_count = len(G)
71     circuits = []
72     psi0 = "0" * qubit_count
73
74     for steps in range(1, d_ind_tot + 1):
75         circuits.append(
76             generate_tfim_circ_custom_graph(
77                 steps, h, J, dt, psi0, G, meas_basis,
78             )
79         )
80     return circuits

```

Estimate single-qubit magnetizations $\langle Z_i \rangle$

To study the dynamics of the model, we want to measure the magnetization of each qubit, defined by the expectation value $\langle Z_i \rangle = \langle \psi | Z_i | \psi \rangle$.

In simulations, we can compute this directly from the measurement outcomes. The function `z_expectation` processes the bitstring counts and returns the value of $\langle Z_i \rangle$ for a chosen qubit index. On real hardware, we evaluate the same quantity by specifying the Pauli operator using the function `generate_z_observables`, and then the backend computes the expectation value.

```

1  def z_expectation(counts, index):
2      """
3      counts: Dict of mitigated bitstrings.
4      index: Index i in the single operator expectation
5      return: < Z_i >
6      """
7      z_exp = 0
8      tot = 0
9      for bitstring, value in counts.items():
10         bit = int(bitstring[index])
11         sign = 1
12         if bit % 2 == 1:
13             sign = -1
14         z_exp += sign * value
15         tot += value
16
17     return z_exp / tot

```

```

1  def generate_z_observables(nq):
2      observables = []
3      for i in range(nq):
4         pauli_string = "".join(["Z" if j == i else "I"
5                                for j in range(nq)])
6         observables.append(SparsePauliOp(pauli_string))
7     return observables

```

```
observables = generate_z_observables(n_qubits)
```

We now define the parameters for generating the Trotterized circuits. In this tutorial, the lattice is a chain of 20 connected triangles, which corresponds to a 40-qubit system.

```

1 | all_circs_mirror = []
2 | for num_triangles in [n_triangles]:
3 |     for meas_basis in ["Z"]:
4 |         A = connected_triangles_adj_matrix(num_triangles)
5 |         G = nx.from_numpy_array(A)
6 |         nq = len(G)
7 |         d_ind_tot = 22
8 |         dt = 2 * np.pi * 1 / 30 * 0.25
9 |         J = 1
10 |        h = -7
11 |        all_circs_mirror.extend(
12 |            trotter_circuits(G, d_ind_tot, J, h, dt, r)
13 |        )
14 | circs = all_circs_mirror

```

Step 3: Execute using Qiskit primitives

Run MPS simulation

The list of Trotterized circuits is executed using the `matrix_product_state` simulator with an arbitrary choice of 4096 shots. The MPS method provides an efficient approximation of the circuit dynamics, with accuracy determined by the chosen bond dimension. For the system sizes considered here, the default bond dimension is sufficient to capture the magnetization dynamics with high fidelity. The raw counts are normalized, and from these we compute the single-qubit expectation values $\langle Z_i \rangle$ at each Trotter step. Finally, we calculate the average over all qubits to obtain a single curve that shows how the magnetization changes over time.

```

1  backend_sim = AerSimulator(method="matrix_product")
2
3
4  def normalize_counts(counts_list, shots):
5      new_counts_list = []
6      for counts in counts_list:
7          a = {k: v / shots for k, v in counts.items()}
8          new_counts_list.append(a)
9      return new_counts_list
10
11
12 def run_sim(circ_list):
13     shots = 4096
14     res = backend_sim.run(circ_list, shots=shots)
15     normed = normalize_counts(res.result().get_counts())
16     return normed
17
18
19 sim_counts = run_sim(circs)

```

Run on hardware

```

1  service = QiskitRuntimeService()
2  backend = service.backend("ibm_marrakesh")
3
4
5  def run_qiskit(circ_list):
6      shots = 4096
7      pm = generate_preset_pass_manager(backend=backend)
8      isa_circuits = [pm.run(qc) for qc in circ_list]
9      sampler = Sampler(mode=backend)
10     res = sampler.run(isa_circuits, shots=shots)
11     res = [r.data.c.get_counts() for r in res.result()]
12     normed = normalize_counts(res, shots)
13     return normed
14
15
16 qiskit_counts = run_qiskit(circs)

```

Run on hardware with Fire Opal

We evaluate the magnetization dynamics on real quantum hardware. Fire Opal provides a Qiskit Function that extends the standard Qiskit Runtime Estimator primitive with automated error suppression and performance

management. We submit the Trotterized circuits directly to an IBM® backend while Fire Opal handles the noise-aware execution.

We prepare a list of `pubs`, where each item contains a circuit and the corresponding Pauli-Z observables. These are passed to Fire Opal's estimator function, which returns the expectation values $\langle Z_i \rangle$ for each qubit at each Trotter step. The results can then be averaged over qubits to obtain the magnetization curve from hardware.

```

1 | backend_name = "ibm_marrakesh"
2 | estimator_pubs = [(qc, observables) for qc in all_circuits]
3 |
4 | # Run the circuit using the estimator
5 | qctrl_estimator_job = perf_mgmt.run(
6 |     primitive="estimator",
7 |     pubs=estimator_pubs,
8 |     backend_name=backend_name,
9 |     options={"default_shots": 4096},
10 | )
11 |
12 | result_qctrl = qctrl_estimator_job.result()
```

Step 4: Post-process and return result in desired classical format

Finally, we compare the magnetization curve from the simulator with the results obtained on real hardware. Plotting both side by side shows how closely the hardware execution with Fire Opal matches the noiseless baseline across Trotter steps.

```

1 | def make_correlators(test_counts, nq, d_ind_tot):
2 |     mz = np.empty((nq, d_ind_tot))
3 |     for d_ind in range(d_ind_tot):
4 |         counts = test_counts[d_ind]
5 |         for i in range(nq):
6 |             mz[i, d_ind] = z_expectation(counts, i)
7 |     average_z = np.mean(mz, axis=0)
8 |     return np.concatenate((np.array([1]), average_z), axis=0)
9 |
10 |
11 | sim_exp = make_correlators(sim_counts[0:22], nq=nq, d_ind_tot=22)
12 | qiskit_exp = make_correlators(qiskit_counts[0:22], nq=nq, d_ind_tot=22)
```

```
1 | qctrl_exp = [ev.data.evs for ev in result_qctrl[:]]  
2 | qctrl_exp_mean = np.concatenate(  
3 |     (np.array([1]), np.mean(qctrl_exp, axis=1)), axis=1  
4 | )
```



```

1  def make_expectations_plot(
2      sim_z,
3      depths,
4      exp_qctrl=None,
5      exp_qctrl_error=None,
6      exp_qiskit=None,
7      exp_qiskit_error=None,
8      plot_from=0,
9      plot_upto=21,
10 ):
11     import numpy as np
12     import matplotlib.pyplot as plt
13
14     depth_ticks = [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
15
16     d = np.asarray(depths)[plot_from:plot_upto]
17     sim = np.asarray(sim_z)[plot_from:plot_upto]
18
19     qk = (
20         None
21         if exp_qiskit is None
22         else np.asarray(exp_qiskit)[plot_from:plot_upto]
23     )
24     qc = (
25         None
26         if exp_qctrl is None
27         else np.asarray(exp_qctrl)[plot_from:plot_upto]
28     )
29
30     qk_err = (
31         None
32         if exp_qiskit_error is None
33         else np.asarray(exp_qiskit_error)[plot_from:plot_upto]
34     )
35     qc_err = (
36         None
37         if exp_qctrl_error is None
38         else np.asarray(exp_qctrl_error)[plot_from:plot_upto]
39     )
40
41     # ---- helper(s) ----
42     def rmse(a, b):
43         if a is None or b is None:
44             return None
45         a = np.asarray(a, dtype=float)
46         b = np.asarray(b, dtype=float)
47         mask = np.isfinite(a) & np.isfinite(b)
48         if not np.any(mask):
49             return None
50         diff = a[mask] - b[mask]
51         return float(np.sqrt(np.mean(diff**2)))
52

```

```

53     def plot_panel(ax, method_y, method_err, color, label):
54         # Noiseless reference
55         ax.plot(d, sim, color="grey", label="Noiseless")
56
57         # Method line + band
58         if method_y is not None:
59             ax.plot(d, method_y, color=color, label=label)
60
61     depths = list(range(d_ind_tot + 1))
62     errors = np.abs(np.array(qctrl_exp_mean) - np.array(sim_exp))
63     errors_qiskit = np.abs(np.array(qiskit_exp) - np.array(sim_exp))
64
65     # Plot RMSE box (vs sim)
66     hi, lo = 0.98, 0.02
67
68     make_expectations_plot(
69         ax,
70         sim_exp,
71         depths,
72         exp_qctrl=qctrl_exp_mean,
73         exp_qctrl_error=errors,
74         exp_qiskit=qiskit_exp,
75         exp_qiskit_error=errors_qiskit,
76         transform=ax.transAxes,
77         ha="center",
78         va="center",
79         fontsize=10,
80         color="0.4",
81     )
82
83     # RMSE box (vs sim)
84     r = rmse(method_y, sim)
85     if r is not None:
86         ax.text(
87             0.98,
88             0.02,
89             f"RMSE: {r:.4f}",
90             transform=ax.transAxes,
91             va="bottom",
92             ha="right",
93             fontsize=8,
94             bbox=dict(
95                 boxstyle="round,pad=0.35", fc="white",
96                 lw=1,
97             ),
98         )
99     # Axes
100    ax.set_xticks(depth_ticks)
101    ax.set_ylim(-1.05, 1.05)
102    ax.grid(True, which="both", linewidth=0.4, alpha=0.5)
103    ax.set_axisbelow(True)
104    ax.legend(prop={"size": 8}, loc="best")

```

References

Tutorial survey

```

105 fig, axes = plt.subplots(1, 2, figsize=(10, 4), c
106
107 axes[0].set_title("Fire Opal (Q-CTRL)", fontsize=
108 plot_panel(
109     axes[0],
110     qc,
111     qc_err,
112     color="#680CE9",
113     label="Fire Opal",
114     band_color="#680CE9",
115 )
116 axes[0].set_xlabel("Trotter step")
117 axes[0].set_ylabel(r"$\langle Z \rangle$")
118 axes[1].set_title("Qiskit", fontsize=10)
119 plot_panel(
120     axes[1], qc, qc_err, color="blue", label="Qis
121 )
122 axes[1].set_xlabel("Trotter step")
123
124 plt.tight_layout()
125 plt.show()

```

Yes



No



© IBM Corp., 2017-2025